RealityOS — Full Project Documentation

> **Simulate the decision before you live it.** *Complete technical and product documentation: what was built, how it is built, every backend source that makes it work, the data model, the API surface, the AI/ML pipeline, the security model, and how to run and deploy it.*

Version: 1.0.0 · Model: `simulated-cohort-v1` (v2) · Compatibility date: `2026-09-01` Last updated: 2026-11-03 · Test suite: 33 automated tests, all passing

---

Table of contents

1. What is RealityOS
2. The core loop (the product)
3. Feature inventory
4. System architecture
5. Technology stack
6. Repository layout
7. Backend: how it works
8. API reference
9. The simulation pipeline
10. The AI layer (agents)
11. The deterministic engine + ML
12. The learning system
13. Database schema
14. Authentication & sessions
15. Security posture
16. Frontend architecture
17. Design system (white, sans, spacious)
18. Configuration & environment
19. Local development, build & deploy
20. Testing
21. Honest limitations
22. System requirements (hardware & software)
23. Module description & test cases
24. Glossary

---

1. What is RealityOS

RealityOS is an **AI decision-simulation platform**. A person describes a real situation in plain language — goals, deadlines, available hours, constraints, resources, competing priorities — and RealityOS:

1. turns it into a **structured decision model**,
2. proposes several **genuinely different strategies**,
3. **simulates** each one with a trained workload model,
4. analyses **risk**, raises **criticisms**, and **recommends** a path,
5. and then, when the real outcome arrives, **scores its own prediction** and calibrates the next one on it.

It is not a chatbot and not a to-do app. Its differentiator is a *closed feedback loop*: the platform makes a falsifiable prediction and then checks itself against reality.

---

2. The core loop (the product)

```
USER INPUT
   -> SITUATION MODEL          (structured understanding of the situation)
   -> DECISION OPTIONS         (3-4 distinct strategies)
   -> SIMULATION               (deterministic engine + trained ML model)
   -> RISK ANALYSIS            (per-option failure modes)
   -> RECOMMENDATION           (ranked, with reasons and why the runner-up lost)
   -> REAL-WORLD OUTCOME       (the user records what actually happened)
   -> PREDICTION VS REALITY    (gaps, causes, Brier score)
   -> LEARNING                 (per-user calibration feeds the next simulation)
```

That loop *is* the product. Every file in the repository serves it.

---

3. Feature inventory

Public website

Page	Route	Purpose
Landing	`/`	Hero, a **live real-engine What-If demo**, the loop explained
How it works	`/how-it-works`	The eight stages of the loop
Features	`/features`	Capabilities + the model card (version, source, AUC)
About	`/about`	What the product is and the design principles
Privacy	`/privacy`	What data is stored, how it is scoped, account deletion
Login	`/login`	Email + password form, or **Continue with Google** (Freebuff OIDC)
Sign up	`/signup`	Same layout; email accounts receive one-time recovery codes
Account recovery	`/forgot-password`	Reset with a recovery code, or recover the provider account
Sign out	`/signout`	Clears the session cookie and shows a confirmation page
404	*	Not found

Application (sign-in required)

Page	Route	Purpose
Dashboard	`/app/dashboard`	Counts, learning snapshot, active simulations
New simulation	`/app/new`	Text + voice + file upload + structured hints
Simulation history	`/app/simulations`	All simulations
Simulation detail	`/app/simulations/:id`	Situation -> options -> comparison -> recommendation -> outcome
Decision profile	`/app/profile`	Personal accuracy, calibration, insights, trend
Settings	`/app/settings`	Theme, model card, training export, account deletion
Onboarding	`/onboarding`	Name, use case, decision style -> one labelled demo simulation

Backend capabilities

- One **Cloudflare Worker** (Hono) serving both the API and the static SPA.

- **D1 (SQLite)** persistence, always scoped by the signed-in user.
- **Server-sent events** for real pipeline progress.
- **Rate limiting**, **same-origin enforcement**, **per-user authorization** on every query.
- **Provider-agnostic LLM layer** with a first-class **deterministic local core** fallback when no key is configured.
- **Three trained models** (two logistic classifiers + one ridge regressor) shipped as weights, explainable per prediction, calibrated per user.

---

4. System architecture

```
+-------------------------------------------------------------------------+
| Browser - React 19 + Vite + Tailwind v4 + motion + Recharts             |
|   same-origin fetch + SSE   -   document parsing happens in-browser    |
+-------------------------------+-----------------------------------------+
                                |  /api/*  and  /auth/freebuff/*
+-------------------------------v-----------------------------------------+
| Cloudflare Worker (single ES-module entrypoint, bundled by esbuild)     |
|                                                                         |
|  index.ts   createFreebuffAuth().wrap()  <- OIDC + PKCE + sessions      |
|  app.ts     origin check -> user resolution -> router -> ASSETS         |
|  routes/api.ts   one route per use case (all authenticated)             |
|                                                                         |
|  services/simulations.ts   lifecycle orchestration + persistence        |
|  services/profile.ts       learning system (reliability, calib.)        |
|  ai/orchestrator.ts        stage pipelines (analyse -> simulate -> )    |
|  ai/agents.ts              7 agents, schema-validated                   |
|  ai/normalize.ts           LLM proposal -> validated domain model       |
|  ai/provider.ts            OpenAI-compatible + Null provider            |
|                                                                         |
|  shared engine (src/shared/engine, imported by BOTH sides)              |
|  shared ML weights (src/shared/ml/weights.ts, auto-generated)           |
|                                                                         |
|  db/repo.ts   all SQL, every statement scoped by owner                  |
|  db/migrate.ts  self-initializing schema (embedded SQL string)         |
|                                                                         |
|  D1 (SQLite)  users profiles simulations situations runs                |
|               predictions outcomes comparisons training_rows            |
|               rate_limits agent_events                                  |
|                                                                         |
|  env.ASSETS   static SPA (run_worker_first: true)                       |
+-------------------------------------------------------------------------+
```

The architecture is deliberately split along three lines:

1. **Deterministic core** — everything numeric, pure, dependency-free, shared by server and browser.
2. **AI layer** — language only (strategies, assumptions, challenges, explanations), never numbers.
3. **Platform layer** — HTTP, auth, persistence, rate limiting: boring and explicit.

---

5. Technology stack

Runtime dependencies (`package.json`)

Package	Version	Role
`@freebuff/auth`	^0.1.0	OIDC + PKCE sign-in, session cookies (server + React)
`hono`	^4.13.13	HTTP router in the Worker
`react` / `react-dom`	^19.3.0	UI
`react-router-dom`	^7.18.4	Client routing + lazy pages
`zod`	^4.6.7	Runtime validation at every boundary
`recharts`	^3.10.1	Charts (lazy chunk)
`motion`	^14.0.0	Animation
`lucide-react`	^1.52.0	Icons
`clsx` + `tailwind-merge`	^2.1.1 / ^3.7.0	Class composition
`pdfjs-dist`	^6.4.299	In-browser PDF text extraction (lazy chunk)
`mammoth`	^1.13.0	In-browser DOCX text extraction (lazy chunk)

Dev tooling

Tool	Role
`vite` ^8.3.3 + `@vitejs/plugin-react`	Frontend bundling & dev server
`@tailwindcss/vite` + `tailwindcss` ^4.3.3	Utility CSS
`typescript` ^7.0.2	`tsc --noEmit` typecheck
`wrangler` ^4.148.0	Local workerd + D1
`esbuild` ^0.28.2	Worker bundle (`tools/build-worker.mjs`)
`@cloudflare/workers-types`	Worker typings

> **Platform deviation (documented honestly):** *the brief suggested Next.js + FastAPI + PostgreSQL. The hosting platform runs* **prebuilt Cloudflare Worker fetch handlers plus static assets** *and provides* **D1 (SQLite)**, *so the shipped stack is a* **Vite/React SPA + a Hono Worker + D1**. *No product behaviour changes, and no SSR adapter or external database is needed. Python/scikit-learn is replaced by an offline TypeScript/Node trainer fitting the same model family (plain gradient descent), keeping everything reproducible in one toolchain.*

---

6. Repository layout

```
src/shared/                 <- shared by Worker AND browser
  types.ts                  domain model (single source of truth for shapes)
  schemas.ts                zod: request bodies, LLM contracts, stored docs
  demo.ts                   the seeded demo scenario
  engine/
    features.ts             workload estimator, capacity, feature vectors, math
    ml.ts                   logistic/ridge inference + MODEL_CARD
    simulate.ts             projections, scoring, recommendation, What-If
    local-engine.ts         deterministic extraction/planning/critique/reflection
    reflect.ts              prediction bookkeeping + outcome comparison
    internals.ts            type-only re-exports (keeps Node type-stripper happy)
  ml/weights.ts             AUTO-GENERATED by `npm run train`

worker/
  index.ts                  entrypoint: auth.wrap() -> app.fetch()
  app.ts                    origin check -> user resolution -> /api -> assets
  env.ts                    bindings + request vars
  routes/api.ts             the whole HTTP surface, one route per use case
  services/simulations.ts   lifecycle orchestration + persistence
  services/profile.ts       learning: readings, calibration, reliability
  ai/provider.ts            OpenAI-compatible provider + Null provider
  ai/prompts.ts             one system prompt per agent
  ai/agents.ts              the seven agents (schema-validated)
  ai/normalize.ts           LLM output -> domain model (resolve, clamp, re-key)
  ai/orchestrator.ts        stage pipelines + per-user calibration
  db/schema.sql             single source of truth for the schema
  db/schema.generated.ts    AUTO-GENERATED embed of the above
  db/migrate.ts             idempotent, self-initializing schema
  db/repo.ts                all SQL, always scoped by owner
  lib/http.ts               JSON/error shape, rate limiting, SSE helper
  lib/logger.ts             structured logging (never logs secrets)

tools/
  train-model.mjs           offline training (imports the runtime feature module)
  build-worker.mjs          esbuild bundle + schema embed
  build-test-worker.mjs     test-only bundle with an injected user resolver

tests/                      engine - agents (mock provider) - end-to-end API
```

7. Backend: how it works

7.1 Entrypoint — `worker/index.ts`

The Worker is one ES module. `createFreebuffAuth(...).wrap()` serves `/auth/freebuff/*` (sign-in, callback, session, sign-out) and keeps sessions fresh *before* the application pipeline sees a request. One auth instance and one app instance are cached per isolate (OIDC discovery and signing keys are cached with them).

```
auth ??= createFreebuffAuth({
  clientId: env.FREEBUFF_CLIENT_ID,
  issuer: env.FREEBUFF_ISSUER,
  baseUrl: env.APP_ORIGIN,          // never derived from the request Host
  allowInsecureLocalhost: env.APP_ORIGIN.startsWith('http://localhost'),
})
```

Unhandled errors are logged structurally and returned as a generic 500 — never a stack trace to the browser.

7.2 Request pipeline — `worker/app.ts`

Order is deliberate:

1. Assign a request id; default `user = null`.
2. **Reject cross-site state-changing calls** — any non-GET/HEAD under `/api/` must pass `isSameOriginRequest(request, APP_ORIGIN)`, otherwise **403**.
3. Resolve the visitor (`getUser`).

4. Route `/api/*` through the authenticated router.
5. Attach `X-Request-Id`, and force `Cache-Control: private, no-store` on API responses.
6. Everything non-API falls through to `env.ASSETS.fetch` (SPA fallback via `not_found_handling: single-page-application`).

`createApp(resolveUser)` is exported separately from the entrypoint so tests can inject a fake user resolver without ever deploying test code.

7.3 HTTP plumbing — `worker/lib/http.ts`

- `AppError` with `code`, `status`, `retryable` -> uniform `ApiError` shape `{ error: { code, message, retryable, requestId } }`.
- `readJson(c, schema)` validates a JSON body through zod and converts failures into friendly field messages.
- `assertSameOrigin` and `enforceRateLimit` guard every mutation.
- `sseResponse(run, c)` streams `StageEvent`s while a pipeline runs and finishes with one `result` event (or an in-band `failed` event).

Rate limiter is a fixed window stored in D1, keyed `user.sub:bucket`:

```
INSERT INTO rate_limits (key, window_start, count) VALUES (?1, ?2, 1)
ON CONFLICT(key) DO UPDATE SET
  count = CASE WHEN rate_limits.window_start = ?2 THEN rate_limits.count + 1 ELSE 1 END,
  window_start = ?2
RETURNING count
```

7.4 API router — `worker/routes/api.ts`

A global `/api/*` guard (everything except `/api/health`):

1. Require a signed-in user -> else **401**.
2. `ensureSchema(DB, requestId)` — a fresh database initializes itself on first authenticated request.
3. `ensureUser(DB, user)` — materialises `users`/`profiles` rows so any endpoint hit first still satisfies foreign keys.
4. `next()`.

Every mutation additionally runs `assertSameOrigin` + `enforceRateLimit`.

7.5 Repository — `worker/db/repo.ts`

All SQL lives here. The rule is absolute: **every statement that touches user data is scoped by** `owner` (= the Freebuff `user.sub`). Persisted JSON is re-validated on the way out (`situationSchema.safeParse`) — stored blobs are never trusted. `deleteAllUserData` removes every table's rows for the owner.

7.6 Schema initialization — `worker/db/migrate.ts`

The hosting platform gives the Worker a D1 binding but no external SQL tool, so the Worker owns its own schema. On first request per isolate it checks `meta.schema_version` and, when behind, applies the idempotent (`CREATE TABLE IF NOT EXISTS`) statements bundled from `worker/db/schema.sql`. That SQL file is the single source of truth for both local development (`npm run db:local`) and production; `tools/build-worker.mjs` embeds it as a string at build time so no filesystem or extra binding is needed at runtime.

8. API reference

All routes are under `/api` and require a session **except** `/api/health`. All mutations must be same-origin. Responses are `private, no-store`. Errors share one shape: `{ error: { code, message, retryable, requestId } }`.

Method	Path	Body / notes	Rate limit
GET	`/api/health`	public: `{ ok, engine, model, time }`	-
GET	`/api/me`	identity + profile + learning snapshot + counts + engine	-
POST	`/api/onboarding`	`{ name, useCase, decisionStyle }` -> seeds demo	20/min
GET	`/api/simulations`	list summaries (<= 60)	-
POST	`/api/simulations`	`createSimulationSchema` -> 201	20/min
GET	`/api/simulations/:id`	full detail (situation, run, outcome, comparison)	-
DELETE	`/api/simulations/:id`	delete own simulation	30/min
POST	`/api/simulations/:id/analyze`	**SSE** -> `{ situation, engine }`	12/min
PATCH	`/api/simulations/:id/situation`	edit the structured situation	60/min
POST	`/api/simulations/:id/run`	**SSE** -> `{ run }`	12/min
POST	`/api/simulations/:id/what-if`	`whatIfSchema` -> projection + deltas	120/min
POST	`/api/simulations/:id/choose`	`{ optionId }`	60/min
POST	`/api/simulations/:id/outcome`	`outcomeSchema` -> comparison + profile	30/min
GET	`/api/simulations/:id/comparison`	comparison + recommendation	-
GET	`/api/profile/decision-pattern`	decision profile	-
GET	`/api/profile/model`	model card + engine summary	-
GET	`/api/profile/training-export`	feature/label rows for retraining	-
POST	`/api/demo`	(re)seed the demo simulation	10/min
POST	`/api/account/delete`	delete all user data	5/min

Request validation

Every body passes a zod schema from `src/shared/schemas.ts` — e.g. a simulation description must be **20-8000 characters**, at most **5 attachments**, at most **12 constraints**, and so on. Invalid input becomes a friendly `400 bad_request`, never a stack trace.

9. The simulation pipeline

`POST .../analyze` and `POST .../run` stream **server-sent events**. Each stage event is emitted when its agent *actually returns*, so the progress list the user sees is real progress, not a timer.

Stage	Agent	Output
Understanding the situation	**Situation**	goal, deadline, hours, active days, constraints, resources, deliverables, risks, open questions, extraction confidence
Extracting constraints / Mapping dependencies	-	derived counts, surfaced as progress detail
Generating decision paths	**Planning**	3-4 strategies with per-deliverable allocations, assumptions, trade-offs, an AI success estimate

Stage	Agent	Output
Simulating outcomes	**Simulation (engine)**	required hours, scheduled hours, expected finish date, buffer, ML completion probability, deadline risk, pressure index, flexibility, utilisation, per-deliverable coverage, per-feature contributions
Identifying risks	**Risk**	option-specific failure modes with severity and mechanism
Challenging assumptions	**Critic**	falsifiable challenges, including explicit disagreement with the model's numbers. Every plan is challenged: when nothing crosses a hard risk threshold, the critic attacks each plan's tightest assumption — its worst-covered deliverable and the buffer left if that estimate runs long
Preparing the recommendation	**Recommendation**	ranking (computed by the engine) + reasons + why the runner-up lost

Numbers are produced only by `src/shared/engine`**.** The LLM's own probability is shown separately, labelled "AI estimate", so the two are never conflated.

---

10. The AI layer (agents)

Provider abstraction — `worker/ai/provider.ts`

RealityOS is not coupled to one vendor: any **OpenAI-compatible chat-completions endpoint** works. When no key is configured, `NoProvider` runs and the platform uses its deterministic local reasoning core instead.

- Model output is always parsed through a zod schema.
- On schema failure or malformed JSON, there is **one repair retry** with the

expected JSON shape appended; nothing downstream ever sees an unvalidated payload.

- `engineInfoFor(provider, startedAt, usedAi, fallbackReason)` reports the engine

that *actually* produced a step — a configured-but-failing provider is never reported to the user as "AI mode".

- The key is read server-side only and never reaches the browser.

The seven agents — `worker/ai/agents.ts`

Each agent has one prompt, one zod output schema and one job. Every agent returns `null` on failure (via `safeAgent`), which makes graceful degradation to the local core a first-class path rather than an error handler.

Agent	Job	Contract
Situation	messy description -> structured decision model	`situationAgentOutputSchema`
Planning	propose distinct strategies	`planAgentOutputSchema`
Simulation	deterministic + ML projections	engine (`simulate.ts`)
Risk	per-option failure modes	`riskAgentSchema`
Critic	attack the plans and the numbers	`critiqueAgentOutputSchema`
Recommendation	explain the computed ranking	`recommendationAgentOutputSchema`
Reflection	predicted vs actual cause analysis	`reflectionAgentOutputSchema`
What-If	explain a variable change	`whatIfExplanationSchema`

Prompts live in `worker/ai/prompts.ts`. They never ask the model for probabilities that get stored — **the ML model owns numbers; the LLM owns language, options and judgement the Critic can challenge.**

Normalization — `worker/ai/normalize.ts`

Model output is treated as a **proposal**. Everything persisted as a number is clamped, cross-checked against the user's own input, and re-keyed to stable ids:

- Deliverables referenced by name are resolved exact-first, then by Dice-coefficient

token overlap **(threshold 0.6); unresolvable allocations are** dropped rather than inventing a deliverable.

- Dependency names are resolved to ids, dropping self-references and cycles.
- The **user's own answers always win** over the model's inference (deadline,

hours, constraints, resources).

- Extraction confidence is bounded by how complete the model actually is.
- Plans with fewer than two usable options return `[]`, triggering the local planner.

Local reasoning core — `src/shared/engine/local-engine.ts`

A deterministic, dependency-free implementation of extraction, planning, critique and reflection. It is not a stub: it produces the *same structure, options and numbers*, just without the LLM narrative layer. Every screen labels which engine is active.

---

11. The deterministic engine + ML

11.1 Workload estimator — `src/shared/engine/features.ts`

Explainable by construction; every hour it attributes is traceable to complexity, dependency count and critical-path status:

```
hours = (2.2 + 6.2*complexity) * (1 + 0.15*min(8, dependencies)) * criticalPathBonus
```

Plus a **coordination overhead** for sequencing and context switching, scaled by fragmentation. Capacity is modelled as:

```
capacityHours = effectiveDays * availableHoursPerDay * reliability * (1 - 0.16*fragmentation)
```

Feature vectors (standardized before the model): `slack, complexity, dependencyDepth, horizon, resourceAdequacy, fragmentation, reliability` (+ `utilization` for the stress model).

11.2 Inference — `src/shared/engine/ml.ts`

Two **logistic regressions** (completion, deadline miss) and one **ridge regressor** (workload pressure), run from shipped weights:

```
successProbability = sigmoid(sum(w_i * z_i) + b)     // clamp 0..1
deadlineRisk       = sigmoid(sum(w_i * z_i) + b)     // clamp 0..1
stressIndex        = clamp(stressLogit, 0.02, 0.99) * 100
```

Every prediction exposes **per-feature signed logit contributions**, so the UI can show *why* the model moved instead of an unexplained percentage.

11.3 Model card (shipped in `src/shared/ml/weights.ts`)

Field	Value
Version	2
Trained at	2026-10-07
Source	`simulated-cohort-v1` (seeded generative cohort, 12,000 samples)

Field	Value
Provenance	"Trained offline on a seeded generative cohort (no real user data). Per-user calibration is applied at runtime from recorded outcomes."
Completion model	accuracy 0.793 - Brier 0.146 - **AUC 0.865** - 2,106 positives
Deadline-miss model	accuracy 0.788 - Brier 0.145 - **AUC 0.872** - 1,833 positives
Stress model	MAE 0.042 - **R2 0.708**

11.4 Projection & recommendations — `src/shared/engine/simulate.ts`

For each option the engine produces: required hours, scheduled hours, per-deliverable coverage (every deliverable is reported, so a silently dropped one shows as 0%), ML completion probability / deadline risk / pressure, a **transparent rule-based feasibility check** shown next to the ML numbers, an expected completion date that accounts for the user's track record, flexibility, utilisation and confidence.

Headline numbers **blend** the trained model (0.7) with the rule check (0.3) so the ML output stays anchored to raw feasibility arithmetic.

Ranking is explicit — nothing is a black box:

```
SCORE_WEIGHTS = { success: 0.40, deadline: 0.18, stress: 0.12,
                  flexibility: 0.12, coverage: 0.18 }
```

`recommend()` returns the winner, numeric reasons, the runner-up comparison, the score and its component breakdown, and a confidence.

Feasibility is stated plainly: `fits` / `tight` / `does_not_fit`, with a note that when the situation does not fit, the only real levers are scope, daily hours or the deadline.

What-If re-projects a single option under overrides (hours/day, active days, horizon, complexity, resource level, focus deliverable) and returns metric deltas plus a deterministic explanation sentence.

The What-If panel maps its four sliders onto those inputs explicitly: a deadline *shift* becomes an ISO deadline, days-per-week maps to `activeDaysPerWeek`, and a complexity haircut scales each deliverable's complexity so the *workload estimate* (not only the ML feature) responds to the lever. Three of those four levers were previously sent as names the API ignores — verified now by a dedicated test.

11.5 Offline training — `tools/train-model.mjs`

```
npm run train                    # train on the simulated cohort
npm run train -- --real data/outcomes.json
```

Because RealityOS has no historical user data on day one, the model is fit on a **generative cohort**: a documented, seeded process model of how work behaves (scope inflation under complexity/dependencies, capacity loss from fragmentation, unplanned overhead from low resource availability, individual follow-through). Labels are *sampled* from that process, so logistic regression must recover the structure from features rather than being told the answer.

Critically, the trainer **imports the runtime's own feature-extraction module** (`src/shared/engine/features.ts`) — training and inference cannot drift apart. `--real` refits the weights on exported outcome data.

---

12. The learning system

This is what makes "RealityOS learns from what actually happened" real rather than decorative. All of it lives in `worker/services/profile.ts` and `src/shared/engine/reflect.ts`.

What a run stores

Each run writes a **prediction** (success probability, deadline risk, pressure, expected hours, expected completion date, deadline) plus the **exact feature vectors** used.

When an outcome is recorded

- The outcome is stored; the gap is computed (`compareWithOutcome`).
- A reflection is produced (LLM or local) with causes and a narrative.
- A `training_rows` entry is appended (features + labels).

Per-user calibration (fed back into the next simulation)

- **Follow-through reliability** — a shrunk estimate

`(sum actual + prior*5) / (n + 5)` starting from the **0.72 population prior**, clamped to `[0.35, 1]`. It becomes a feature of the next simulation.

- **Bias correction** — `mean(actual - predicted)` applied with shrinkage

`n/(n+5)`, so one surprising outcome cannot swing the model.

- The **decision profile** computes prediction accuracy, time-estimation

accuracy, deadline reliability, risk tendency, average stress, a **Brier score** (lower is better), data-backed insights, and an accuracy-over-time trend.

Retraining loop

`GET /api/profile/training-export` returns the rows in the trainer's input format; `npm run train -- --real <file>` refits the weights on them.

---

13. Database schema

Cloudflare D1 / SQLite. Every user-owned row carries `owner` = the Freebuff `user.sub`; queries are always scoped by owner. Source: `worker/db/schema.sql`.

Table	Key columns	Purpose
`users`	`sub` (PK), email, name, picture, timestamps	Identity mirror
`profiles`	`sub` (PK -> users), use_case, decision_style, onboarded_at	Onboarding + preferences
`simulations`	`id` (PK), owner, title, status, description, is_demo, goal, deadline, top_ metrics, risk_level, confidence	One decision per row
`situations`	`id`, simulation_id, owner, `payload` (JSON), edited	Versioned structured situation snapshots
`runs`	`id`, simulation_id, owner, `payload` (JSON), engine_mode	Options, projections, comparison, recommendation
`predictions`	`id`, simulation_id, owner, success_probability, deadline_risk, stress_index, expected_hours, expected_completion_date, deadline	The falsifiable part
`outcomes`	`id`, simulation_id, owner, option_id, `payload` (JSON)	What actually happened
`comparisons`	`id`, simulation_id, owner, `payload` (JSON)	Prediction-vs-reality
`training_rows`	`id`, owner, simulation_id, `features` (JSON), success, deadline_miss, stress01	Retraining export
`rate_limits`	`key` (PK), window_start, count	Fixed-window limiter
`agent_events`	`id`, simulation_id, owner, agent, status, detail, duration_ms	Audit trail of agent activity

Indexes: `simulations(owner, created_at DESC)`, and `(simulation_id, created_at DESC)` on `situations`, `runs`, `predictions`, `outcomes`.

`meta(key, value)` is created by the migration and stores `schema_version = '1'`.

Simulation status lifecycle: `draft -> analyzed -> simulated -> decided -> completed`.

---

14. Authentication & sessions

- **Sign in with Freebuff** — OpenID Connect + PKCE, presenting Google, GitHub and Apple. RealityOS never sees or stores a password.
- `worker/index.ts` wraps everything with `createFreebuffAuth(...).wrap(...)`, so `/auth/freebuff/*` (sign-in, callback, session, sign-out) is handled before the app and **sessions are refreshed with the cookies the SDK returns**.
- The browser uses `useFreebuffUser()` (from `@freebuff/auth/react`) for identity state and `SignInWithFreebuff` for the button; `safeReturnTo` validates the post-login redirect.
- Sessions use **secure, HTTP-only cookies**. `baseUrl` is `APP_ORIGIN` and is **never derived from a request Host**.
- `allowInsecureLocalhost` is enabled **only** for loopback development.
- Registered callback path: `/auth/freebuff/callback`.

The login page is intentionally minimal: one card with an email + password form as the normal path, a divider, then "Continue with Google". Both doors land in the same account (`sub` is the identity either way), so a user can add or switch without losing simulations. Sign-up, recovery and sign-out share the layout.

Email + password accounts (owned by the Worker):

- Passwords are hashed with PBKDF2-SHA256, 210,000 iterations and a per-account salt through WebCrypto; the plain text is never stored or logged.
- Sessions are random 256-bit tokens in an **HTTP-only, SameSite=Lax cookie** (Secure on HTTPS); only the SHA-256 of the token is stored in `sessions`, so a database leak cannot be replayed.
- Sign-up issues eight **one-time recovery codes** (shown once, stored hashed). A code resets the password, signs out every other device, and cannot be reused.
- Credential endpoints are rate limited per email address and same-origin checked, and every response is `private, no-store`.
- `email_edited` protects a chosen address from being overwritten by a later provider refresh.

---

15. Security posture

Control	Implementation
Authorization	Server-side on every user-scoped query (`owner = user.sub`); client-supplied ids decide nothing
Ownership re-check	Mutations re-check ownership **before** parsing the body, so someone else's id returns 404 regardless of payload validity
CSRF	State-changing requests must be **same-origin**, checked before authentication runs
Rate limiting	Fixed-window per user per endpoint in D1 (analyze/run 12/min, what-if 120/min, ...)
Caching	All API responses are `Cache-Control: private, no-store`
Input validation	Every request body zod-validated at the boundary

Control	Implementation
Secret handling	LLM keys live in server-side secrets; the browser never receives them. **No secrets in the repo**
Uploads	Documents are parsed **in the browser**; only extracted text is sent, and images are never read for text. No file-storage binding -> that attack surface is removed entirely
Account deletion	Removes profile, simulations, situations, runs, predictions, outcomes, comparisons, training rows and agent events for that user
Logging	Structured; **never logs cookies, tokens or auth URLs**
Error handling	Generic 500 to the client; details only in server logs

---

16. Frontend architecture

A React 19 + Vite single-page app (TypeScript, `strict`). It is deliberately thin: all business logic that must be identical in both places lives in `src/shared/*`, which the browser imports directly — so the landing page can run the real projection engine with no server round-trip.

16.1 Entry, routing & guards

- **Entry** — `src/main.tsx` mounts React in `StrictMode` inside `BrowserRouter`.
- **Routing** — `src/App.tsx` uses `react-router-dom` with **lazy-loaded pages**

(code splitting) and a `Suspense` fallback:

- `MarketingLayout` (navbar + footer) wraps the public pages.
- `RequireAuth` guards the `/app/*` routes and never redirects mid-flight

(it shows a spinner while checking, then a sign-in prompt).

- `/app` nests `AppShell` (sidebar + topbar) around the app pages.
- A `ScrollToTop` helper resets scroll on navigation; `*` renders `NotFound`.

16.2 Page reference

Route	Component file	Responsibility
`/`	`pages/public/Landing.tsx`	Hero, the live in-browser What-If demo, the loop explained
`/how-it-works`	`pages/public/HowItWorks.tsx`	The eight stages of the decision loop
`/features`	`pages/public/Features.tsx`	Capability grid, engine modes, the model card (AUC)
`/about`	`pages/public/About.tsx`	Philosophy, non-goals, audience, **full documentation download**
`/privacy`	`pages/public/Privacy.tsx`	What is stored, how it is scoped, how to delete it
`/login`	`pages/public/Login.tsx`	One action: continue with Freebuff (`AuthShell`)
`/signup`	`pages/public/SignUp.tsx`	Create the account (`AuthShell`)
`/forgot-password`	`pages/public/ForgotPassword.tsx`	Password recovery guidance (`AuthShell`)
`/onboarding`	`pages/app/Onboarding.tsx`	Capture use case + decision style, seed the demo simulation
`/app/dashboard`	`pages/app/Dashboard.tsx`	Overview: recent simulations, learning snapshot, next steps
`/app/new`	`pages/app/NewSimulation.tsx`	Capture the situation (+ attachments), then analyse

Route	Component file	Responsibility
`/app/simulations`	`pages/app/SimulationHistory.tsx`	Every simulation, with status and filters
`/app/simulations/:id`	`pages/app/SimulationDetail.tsx`	Analysis, comparison, What-If, choose, record outcome
`/app/profile`	`pages/app/DecisionProfile.tsx`	Decision patterns, reliability, calibration, model info
`/app/settings`	`pages/app/Settings.tsx`	Theme, export, account deletion
`*`	`pages/public/NotFound.tsx`	404

16.3 Client libraries (`src/lib/*`)

Module	Responsibility
`api.ts`	The only door to the backend: a typed `fetch` client with uniform error handling (`RequestError` carrying `code`, `status`, `retryable`, `requestId`) plus **SSE-over-fetch** readers (`streams.analyzeSimulation`, `streams.runSimulation`) that surface real stage events
`session.tsx`	`SessionProvider` / `useSession`: merges Freebuff identity (`useFreebuffUser`) with the `/api/me` payload (profile, learning, engine, counts) and ows the theme (`light \
`documents.ts`	In-browser extraction from PDF / DOCX / plain text (`pdfjs-dist`, `mammoth`) — heavy parsers load lazily
`useAttachments.ts`	Attachment lifecycle (add, parse, preview, remove) for the situation form
`voice.ts`	Optional speech-to-text input for dictating the situation
`hooks.ts`	Small shared hooks (media query, debounce, etc.)
`format.ts`	Number/date/percent formatting used across the UI
`demo.ts`	Client-side helpers for the seeded demo scenario

16.4 Component library (`src/components/*`)

Group	Exports
`ui/`	`Button` (`Button`, `ButtonLink`, `IconButton`), `Card` (`Card`, `CardHeader`, `CardTitle`, `CardBody`, `Section`), `Badge`, `Modal`, `Toast` (`ToastProvider`), `Form`, `Feedback` (`Spinner`, empty/error states), `Metric`
`layout/`	`AppShell` (sidebar + topbar), `AuthShell`, `Navbar`, `Footer`, `Logo`, `PageHeader`
`app/`	`SituationPanel`, `ComparisonPanel`, `RecommendationPanel`, `OutcomePanel`, `WhatIfPanel`, `DecisionCard`, `ConfidenceRing`, `SimulationCard`
`charts/`	`Charts` — `recharts` wrappers themed to the design tokens
`marketing/`	`HeroPipeline`, `LiveWhatIf` — the animated story and the real-engine demo
`ambient/`	`AmbientBackground` — dark-mode-only particle/glow field

16.5 Data flow & state

1. On load, `useFreebuffUser()` calls `/auth/freebuff/session`; if signed in, `SessionProvider` then calls `/api/me` for profile, learning and engine data.
2. Pages read the session from context and call `src/lib/api.ts`; nothing in the app talks to the backend directly.
3. `analyze` and `run` open an SSE stream: the UI paints each **real** stage event (understand → plan → simulate → risk → critique → recommend → reflect) as the server emits it, then applies the final `result` payload.
4. Engine math for the landing-page demo is imported from `src/shared/engine/*` and runs entirely in the browser — no network, identical numbers.

16.6 Bundling & dev proxy

- **Alias** — `@ -> ./src`.
- **Output** — `dist/client`, target `es2022`, no sourcemaps in production.
- **Manual chunks** — `pdf` (`pdfjs-dist`), `docx` (`mammoth`), `charts` (`recharts/d3-*`) and `vendor`; the document parsers are pulled in only when a file is actually attached.
- **Dev proxy** — Vite (port `3000`, host `0.0.0.0`) proxies `/api` and `/auth/freebuff` to the local Worker on `http://127.0.0.1:8787`, so the browser sees a single origin in development. Because the proxy preserves the browser's `Origin` (`http://localhost:3000`) while the Worker's `APP_ORIGIN` is `http://localhost:8787`, the local config also sets `DEV_ORIGIN` to the Vite dev origins (a comma-separated list). State-changing calls are accepted from the canonical origin **or** that explicitly configured dev origin; any other origin is still rejected with 403.

---

17. Design system (white, sans, spacious)

Defined in `src/styles/index.css` as Tailwind v4 `@theme` tokens, with two palettes switched by a class on `<html>`:

- **Light (product default):** white page, near-white panels, dark ink text, one electric accent. `index.html` ships `<html class="light">`.
- **Dark (opt-in via the header toggle):** deep navy surfaces, soft mist text.

Key decisions from the latest redesign:

- **Sans typography everywhere.** `--font-sans` is Inter (loaded via Google Fonts), falling back to system UI fonts. The numeric `.num` style uses **tabular numerals in the sans stack**, not monospace.
- **Clean and spacious.** Generous vertical rhythm, one-column auth pages, single cards, one primary action.
- **Restrained decoration.** The background is white plus a barely-there wash; the particle field, drifting colour blobs and grid are **dark-mode only**.
- **Semantic colour.** The accent marks meaning (primary action, recommendation, active state); violet marks the second engine only; `signal-low/medium/high` express risk.

---

18. Configuration & environment

Copy `.env.example` to `.dev.vars` for local work. On the hosting platform the reserved auth variables are injected automatically; anything you add goes in **Sites -> Secrets**.

Variable	Purpose
`FREEBUFF_CLIENT_ID`	Public OIDC client id (injected on deploy)
`FREEBUFF_ISSUER`	OIDC issuer, `https://freebuff.com/oidc`
`APP_ORIGIN`	Canonical origin of the deployed site (never from Host)
`AI_PROVIDER`	`auto
`AI_API_KEY`	**Secret.** Any OpenAI-compatible key. Absent -> local core runs
`AI_BASE_URL`	Default `https://api.openai.com/v1`
`AI_MODEL` / `AI_VISION_MODEL`	Model ids, e.g. `gpt-4o-mini`

Variable	Purpose
DEV_ORIGIN	**Local development only.** A comma-separated list of dev origins (http://localhost:3000,http://127.0.0.1:8787) permitted to send state-changing requests through the dev proxy; compared by host because Vite forwards its own scheme. Never set in deployment

No secrets belong in the repository. The LLM key is read server-side only.

---

19. Local development, build & deploy

Commands

```
npm install                     # Node 20+ (developed on Node 24)

npm run train                   # refit the workload model -> src/shared/ml/weights.ts
npm run db:local                # apply worker/db/schema.sql to local D1
npm run dev:worker              # Worker + API on http://localhost:8787
npm run dev                     # Vite dev server on http://localhost:3000 (proxies /api)

npm run typecheck               # tsc --noEmit across app, worker and tooling
npm test                        # engine + agent tests (node --test)
node --test tests/api.test.mjs  # end-to-end API tests in real workerd with real D1
```

`npm run build` runs `build:web` then `build:worker`:

- `dist/client` — the SPA (Vite).
- `dist/worker/index.js` — the bundled Worker (esbuild, `platform: neutral`, workerd-first resolution, minified, **sourcemaps off**, schema embedded).

Wrangler config (`wrangler.jsonc`)

```
{
  "name": "realityos",
  "main": "dist/worker/index.js",
  "compatibility_date": "2026-09-01",
  "assets": {
    "directory": "./dist/client",
    "binding": "ASSETS",
    "run_worker_first": true,
    "not_found_handling": "single-page-application"
  },
  "d1_databases": [{ "binding": "DB", "database_name": "realityos" }],
  "observability": { "enabled": true }
}
```

Deployment

The site is deployed through the Freebuff **Sites** hosting pipeline (prebuilt Worker + static assets, D1 binding attached on deploy). Current URLs are recorded in `site.json`:

Environment	URL
Production	https://site-7fbfe404b2c14db4af9e943b0c4a1672.freebuff.page
Preview	https://site-7d7d02b3aa1842cd85be2675f6e0a4da.freebuff.page

---

20. Testing

Three layers, all run with Node's built-in test runner.

`tests/engine.test.mjs` (unit / property level, 12 tests)

- extraction of dates, hours and deliverables
- determinism (same inputs => same numbers)
- monotonicity (more hours => higher completion probability, lower deadline risk, higher pressure)
- scoring weights summing to 1
- What-If delta direction
- **every What-If lever the UI exposes changes the projection** (earlier deadline, fewer active days, lower complexity)
- prediction-vs-reality arithmetic

`tests/agents.test.mjs` (AI layer, 6 tests)

- LLM branch with a **mock provider**: normalization, hallucination rejection, schema-violation fallback, engine-mode honesty, stage events

`tests/api.test.mjs` (end-to-end, 15 tests)

- the real Worker bundle in **real workerd** with a **real local D1**, driven over HTTP: 401s, foreign-origin 403s, two-account isolation, validation errors, the complete `create -> analyze -> run -> what-if -> choose -> outcome -> profile` loop, calibration, training export and account deletion

33 tests in total (12 engine + 6 agent + 15 API), all passing against the current build, alongside `tsc --noEmit` and a production build.

`tests/api.test.mjs` also drives the email account surface: sign-up with recovery codes, sign-in (case-insensitive email), wrong-password 401, profile name + email updates, password change requiring the current password, one-time recovery codes, sign-out invalidating the session, and foreign-origin 403s on the credential endpoints.

> *If no LLM key is configured, nothing pretends otherwise: every screen labels the active engine, and the local core produces the same structure, options and numbers.*

---

21. Honest limitations

- The workload model is trained on a **seeded generative cohort**, not on real user data. The cohort process, its assumptions and its held-out metrics are documented in `tools/train-model.mjs` and surfaced in the UI (Features and Settings show version, source and AUC). Per-user calibration comes from outcomes the user actually records, and `--real` refits the weights on exported data.
- **Simulations are estimates, not guarantees.** The UI says so and never presents a probability as a promise.
- **Sign-in identity is owned by the Freebuff provider.** The email/verification step some users see belongs to Freebuff's own account flow, not RealityOS. Sign-in that skips it entirely would require a separate Google OAuth client and new `/auth/google/*` routes in the Worker.
- The deployed site depends on the hosting platform's availability; platform-level outages appear as `Site temporarily unavailable` on every route, including static assets.

---

22. System requirements (hardware & software)

22.1 To use RealityOS (end user)

RealityOS is a website: nothing is installed and no minimum specification is enforced. These are the practical requirements.

Component	Requirement
Device	Any modern laptop, desktop, tablet or phone
Processor	Dual-core or better; an Intel Core i3/i5 or equivalent is comfortable
RAM	4 GB minimum, 8 GB recommended
Storage	No installation; a few MB of browser cache
GPU	**Not required** — no GPU work happens in the browser or on the server
Display	Any resolution; the layout is responsive from ~360 px upward
Input	Keyboard and pointer; a microphone only for the optional dictation feature
Browser	Current Chrome, Edge, Firefox or Safari (the bundle targets ES2022)
Internet	Required for sign-in, saving simulations and the API. The landing-page What-If demo runs entirely in the browser

22.2 To develop, test, build or train

Component	Requirement
Operating system	Windows 10/11, macOS or Linux
Runtime	**Node.js 20 or newer** (developed on Node 24) and npm
Memory / disk	8 GB RAM recommended; ~1 GB for dependencies, build output and the local database
GPU / ML toolchain	**None.** No CUDA, TensorFlow, scikit-learn, Jupyter or Anaconda — the model is trained offline by a plain-gradient-descent script in Node (`npm run train`)
Editor	Any TypeScript-capable editor (VS Code used throughout)
Local runtime	Wrangler / workerd (installed as a dev dependency) for the Worker and D1
Hosting (optional)	A Cloudflare account with Workers and D1 permissions to self-host, or the managed Freebuff Sites pipeline
AI (optional)	An OpenAI-compatible API key for the language-model agents; without it the deterministic local core runs

22.3 Software stack summary

Layer	Technology
Front end	React 19, TypeScript, Vite 8, React Router 7, Tailwind CSS v4, Motion, Recharts, lucide-react
Back end	Cloudflare Worker (one ES-module entrypoint) with Hono routing; server-sent events for pipeline progress
Database	Cloudflare D1 (managed SQLite); 14 tables, self-initializing schema with forward migrations
Authentication	Sign in with Freebuff (OpenID Connect + PKCE) or an email + password account (PBKDF2-SHA256, 210k iterations, one-time recovery codes); HTTP-only session cookies, no plain-text passwords
Validation	Zod schemas at every boundary (request bodies, LLM output, stored JSON)
AI (optional)	Any OpenAI-compatible chat-completions provider, behind a provider abstraction with a local fallback
ML	Two logistic regressions (completion, deadline miss) + one ridge regressor (workload pressure), trained offline and shipped as weights
Document parsing	`pdfjs-dist` and `mammoth`, running in the browser only
Testing	Node's built-in test runner (`node --test`); real workerd + real local D1 for the API suite
Tooling	esbuild (Worker bundle), `tsc --noEmit`, Wrangler, Tailwind CLI through Vite

Layer	Technology
Hosting	Freebuff Sites (managed Worker + D1), or the self-hosted Cloudflare config in `wrangler.deploy.jsonc`

23. Module description & test cases

23.1 Modules

#	Module	Files	Responsibility
M1	Situation intake	`src/pages/app/NewSimulation.tsx`, `src/lib/documents.ts`, `src/lib/useAttachments.ts`, `src/lib/voice.ts`	Capture the decision in text, by dictation or by attached PDF/DOCX/text parsed in the browser
M2	Situation model	`src/shared/engine/local-engine.ts`, `worker/ai/agents.ts`, `worker/ai/normalize.ts`, `src/components/app/SituationPanel.tsx`	Extract and let the user correct goal, deadline, hours, deliverables, dependencies, constraints, resources, risks
M3	Decision planning	`worker/ai/agents.ts` (Planning), `planOptionsLocally`	Produce 3–4 materially different strategies with allocations, assumptions and trade-offs
M4	Simulation engine	`src/shared/engine/features.ts`, `ml.ts`, `simulate.ts`	Workload/capacity modelling, feature extraction, ML inference, projections, feasibility, scoring, ranking, What-If deltas
M5	Risk & critic	`optionRisks` in `simulate.ts`, `critiqueLocally`, Risk and Critic agents	Per-option failure modes and falsifiable challenges to the plan's assumptions
M6	Outcome & learning	`src/shared/engine/reflect.ts`, `worker/services/profile.ts`	Record what happened, score the prediction, compute reliability/bias calibration, insights and trend
M7	API & auth	`worker/index.ts`, `worker/app.ts`, `worker/routes/api.ts`, `worker/lib/http.ts`	OIDC sessions, same-origin enforcement, rate limiting, 19 endpoints, SSE streaming, uniform error shape
M8	Persistence	`worker/db/schema.sql`, `migrate.ts`, `repo.ts`	12-table schema, self-initializing migration, every query scoped by owner
M9	Machine learning	`tools/train-model.mjs`, `src/shared/ml/weights.ts`	Offline training on a seeded cohort on the runtime's own feature module; retraining on exported real rows
M10	Public website	`src/pages/public/*`, `src/components/marketing/*`	Landing (with the live in-browser demo), How it works, Features, About, Privacy, auth pages, 404
M11	Signed-in workspace	`src/pages/app/*`, `src/components/app/*`, `src/components/layout/AppShell.tsx`	Dashboard, history, detail (situation → options → comparison → recommendation → outcome), profile, settings
M12	Design system	`src/styles/index.css`, `src/components/ui/*`	Tailwind v4 theme tokens, light/dark palettes, buttons, cards, modals, toasts, forms, charts, feedback states

23.2 Automated test cases

ID	Module	Scenario	Expected result
TC-01	M2	Prose with “October 15”, “in 5 days”, “next Friday”, “friday”	Each phrase resolves to the correct ISO deadline; unknown text returns `null`
TC-02	M2	Hours phrased per day and per week	Correct `availableHoursPerDay` / active days

ID	Module	Scenario	Expected result
TC-03	M4	Same situation and option projected twice	Identical numbers (determinism)
TC-04	M4	More daily hours	Completion probability and coverage rise; deadline risk falls; pressure rises
TC-05	M4	Longer horizon	Scheduled hours scale with the capacity model
TC-06	M4	Score every option	Scores in range, five weighted components summing to 1, recommendation matches the top score
TC-07	M4	What-If with 1 h/day instead of ~4	Delta reported with `direction: worse` plus a deterministic explanation
TC-08	M4	What-If with an earlier deadline, fewer active days, lower complexity	Risk rises / completion falls / required hours fall accordingly
TC-09	M6	“Partly” completed after the predicted date	Negative success delta, hours error, missed deadline, causes and narrative
TC-10	M5	Deliberately over-committed options	At least one critique, referencing capacity/buffer/coverage
TC-11	M5	Comfortable, feasible options	Every plan still receives its tightest-assumption challenge
TC-12	M7	Unsigned request to `/api/me`, `/api/simulations`, `/api/profile/*`	401 with `error.code = unauthorized`
TC-13	M7	Mutation with a foreign `Origin`	403 with `error.code = forbidden`
TC-14	M7	Account B reads/mutates account A's simulation	404, and A's data is unchanged
TC-15	M7	20-character-minimum input violated	400 with a friendly, actionable message
TC-16	M7	`run` before `analyze`	409 `not_analyzed`, never a crash
TC-17	M1–M8	Full loop: create → analyze (SSE) → run (SSE) → What-If → choose → outcome → profile	Every stage streams, ≥4 stage events, ≥3 options, recommendation with ≥2 reasons, comparisons, calibration and a training export
TC-18	M8	Account deletion	Every row for that user is gone
TC-19	M9	`npm run train`	Weights and model card regenerate; metrics reported
TC-20	M10/M11	Render every route	All 16 routes render without an exception

23.3 Manual verification performed

Check	Result
`npm run typecheck` (`tsc --noEmit`)	Passes
`npm run build` (SPA + Worker bundle)	Passes; `dist/client` + `dist/worker/index.js`
`npm test`	27/27 passing
Public routes served by the Worker and by Vite	200 for every route
`/api/health`	200 `{ ok: true, engine, model }`
Signed-out `/api/*`	401
Mutation from a foreign origin	403
Account flow through the Vite dev proxy (`DEV_ORIGIN`)	Sign-up 201 with cookie, `/api/me` 200, profile edit 200, sign-out 200, then 401
`/favicon.svg`, `/RealityOS-Project-Documentation.pdf`	200 with correct content types
Interactive checks (slider drag, modals, sign-in)	Require a signed-in browser session — see section 21

24. Glossary

Term	Meaning
Situation	The structured model of the user's decision context
Deliverable	A unit of work the goal decomposes into
Option	One candidate decision path with per-deliverable allocations
Projection	The simulated outcome of an option (probabilities, hours, dates, coverage)
Evaluation	An option plus its projection plus its risks
Prediction	The stored, falsifiable headline numbers for a run
Outcome	What the user records actually happened
Comparison	Prediction-vs-reality with gaps and causes
Reliability	Per-user shrunk follow-through rate fed into the model
Calibration	Per-user shrunk bias correction on headline probabilities
Engine mode	`ai` (LLM narrative active) or `local` (deterministic core)